Smart Asset Management & Resource Allocation Platform

(SAMRAP)

System Architecture & Design Specification

Author: Pratyush Mehta

1. Problem Understanding

Efficient physical asset management poses significant operational challenges in medium-to-large organizations. Shared assets—such as cameras, studio lighting, microphones, costumes, and event props—are frequently requested by different users and team members for overlapping durations. In the absence of a centralized system, organizations face severe challenges:

- **Double-Bookings & Conflicts:** Simultaneous requests for the same physical asset units.
- **Lack of Accountability:** Insufficient logging of who checked out which item, when it was returned, and its condition.
- **Stock Underutilization:** Assets remain idle because potential borrowers are unaware of their availability.
- **Inefficient Lifecycle Auditing:** Lack of structural health logs, making it hard to track repairs, damages, or retirement cycles.

SAMRAP addresses these vulnerabilities directly. By introducing role-based access controls, a transaction-safe overlap checker, automated static QR code utilities, condition logs, and a dynamic analytics dashboard, SAMRAP centralizes resource coordination and maintains database state integrity.

2. System Architecture

SAMRAP is designed as a monolithic full-stack application using the Django web framework. The design prioritizes modularity, performance, and self-containment for rapid deployment.

Component	Description & Tech Choice
Presentation Layer	Django HTML5 templates styled with Vanilla CSS Variables. Renders a modern, responsive glassmorphic dark theme. Icons by Lucide. Analytics by Chart.js.
Logic / API Controller	Django Class-Based and Functional Views. Handled via secure URL routing, Forms engine, and Session-based Authentication middleware.
Data Layer	SQLite database file (db.sqlite3). Ported relational store with strict foreign keys and constraints.
QR Processing	Python qrcode library. Generates base64 QR PNG strings natively, rendered in views without file writes.

3. Database Schema Specification

The relational database schema is structured to ensure referential integrity, automated constraints, and audit capability:

3.1 User Model (AbstractUser Extension)

Extends Django's authentication model to define user access roles.

```
• username (CharField, unique)
• email (EmailField, unique)
• password (CharField, hashed)
• name (CharField, full name)
• role (CharField: 'ADMIN' or 'USER')
```

3.2 Category & Asset Models

Tracks physical equipment metadata and inventory levels.

```
Category:
• id (UUID / Auto Field)
• name (CharField, unique)
• description (TextField)

Asset:
• id (UUID / Auto Field)
• name (CharField)
• category (ForeignKey to Category)
• total_qty (Positive Integer, defaults to 1)
• status (CharField choices: READY, MAINTENANCE, DAMAGED, RETIRED)
• qr_code_url (TextField: contains base64 image data)
```

3.3 Booking, Health, and Audit Models

Tracks lending states, repairs history, and security logs.

```
Booking:
• user (ForeignKey to User) | asset (ForeignKey to Asset)
• quantity (Positive Integer)
• start_date, end_date (DateTimeField)
• status (PENDING, APPROVED, REJECTED, CANCELLED, ISSUED, RETURNED)
• issued_at, returned_at (DateTimeField, nullable)

AssetHealth:
• asset (ForeignKey to Asset) | condition (Good, Fair, Damaged, Unusable)
• notes (TextField) | created_at (DateTimeField)

AuditLog:
• user (ForeignKey to User, null=True) | action (CharField)
• details (TextField / JSON) | created_at (DateTimeField)
```

4. Entity Relationship Diagram (ERD)

Below is a structural schematic mapping table primary keys, foreign keys, and relationships:

Table Relation	Relationship Type	Key Constraints
User → Booking	1 to Many	Booking.user_id = User.id
Asset → Booking	1 to Many	Booking.asset_id = Asset.id
Category → Asset	1 to Many	Asset.category_id = Category.id
Asset → AssetHealth	1 to Many	AssetHealth.asset_id = Asset.id
User → AuditLog	1 to Many (Optional)	AuditLog.user_id = User.id (SET_NULL)

5. API & URL Routes Mapping

The routing architecture maps to specific Django views with permission gate decorators:

Endpoint Route	Method	Access	Description
/register/	GET/POST	Public	User registration page.
/login/	GET/POST	Public	Session logging page.
/assets/	GET	Auth	Asset catalog & search.
/assets/add/	GET/POST	Admin	Registers a new asset.
/bookings/request/<id>/	POST	User	Places a reservation request.
/approvals/	GET	Admin	Admin reviews pending requests.
/bookings/<id>/action/<act>/	POST	Role-Based	Approve, reject, issue, or return.
/qr/scan/<id>/	GET/POST	Auth	Decodes tag, quick checkout/in.
/analytics/	GET	Admin	Renders Chart.js visuals.

6. Key Design Decisions

6.1 Choice of SQLite

We chose SQLite because it is a lightweight, zero-configuration file-based database that ships with Python. This completely eliminates setup errors for evaluation environments, while fully supporting relational constraints, foreign key actions, and thread-safe operations in write-ahead logging (WAL) mode.

6.2 Choice of Django

Django was selected over a decoupled React-Express stack for multiple reasons: Built-in secure session management, robust form validation modules, built-in ORM with transactions support, and simple monolithic file-structure. This eliminates CORS issues, multiple package.json configurations, and deployment complexity.

6.3 Overlap Checker & Concurrency

To prevent stock allocation conflicts, we implemented a timeline sweep algorithm wrapped inside transaction.atomic() block combined with select_for_update(). When a user requests a booking, the database locks the row, finds all overlapping active bookings in that time-frame, computes the peak concurrent allocation, and checks it against total inventory before approving.

6.4 Dynamic base64 QR Engine

Rather than saving generated QR codes to media directories (creating file paths management overhead), the QR engine encodes routing URLs directly into base64 data URLs. This is lightweight, easily rendering inside standard HTML <img> tags, and facilitates easy Docker mounting.